TP 4 : Régression linéaire & Descente de Gradient

ATTIA Oussama Abderraouf - SIL1

12 mars 2026

Table des matières

1 Exercice 1 : Régression Linéaire **2**
1.1 1–2. Nuage de points et Courbe Lowess 2
1.2 3–4. Modèle de Régression Linéaire Classique 2
1.3 5–9. Transformation Logarithmique 3

2 Exercice 2 : Descente de Gradient **5**
2.1 1. Implémentation 5
2.2 2–3. Convergence et Itérations nécessaires 5
2.3 4. Ajout de la Régularisation (Pénalité L2 - Ridge) 6

1 Exercice 1 : Régression Linéaire

1.1 1–2. Nuage de points et Courbe Lowess

Le jeu de données `cars` contient 50 observations de la vitesse d'une voiture (`speed`) et de la distance requise pour s'arrêter (`dist`).

```
plot(cars, xlab="Speed (mph)", ylab="Stopping distance (ft)", las
    =1)
lines(lowess(cars$speed, cars$dist, f=2/3, iter=3), col="red")
```

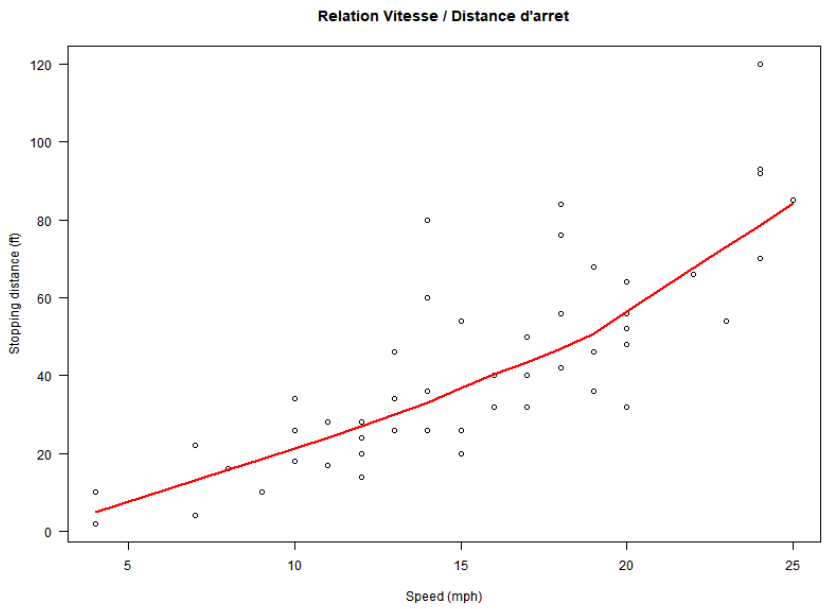

FIGURE 1 – Relation Vitesse / Distance d'arrêt (Courbe Lowess en rouge)

Constat sur la forme de la courbe : La courbe `lowess` n'est pas parfaitement droite. Elle démarre de façon assez linéaire pour les faibles vitesses, puis s'incurve (croissance exponentielle ou quadratique) vers le haut lorsque la vitesse augmente. Cela suggère que la distance de freinage augmente plus vite que proportionnellement par rapport à la vitesse.

1.2 3–4. Modèle de Régression Linéaire Classique

On effectue la régression linéaire de la vitesse (`speed`) en fonction de la distance (`dist`). *Note : Bien que physiquement la distance dépende de la vitesse, on suit la formule de l'énoncé.*

```
reg = lm(speed ~ dist, data=cars)
summary(reg)
```

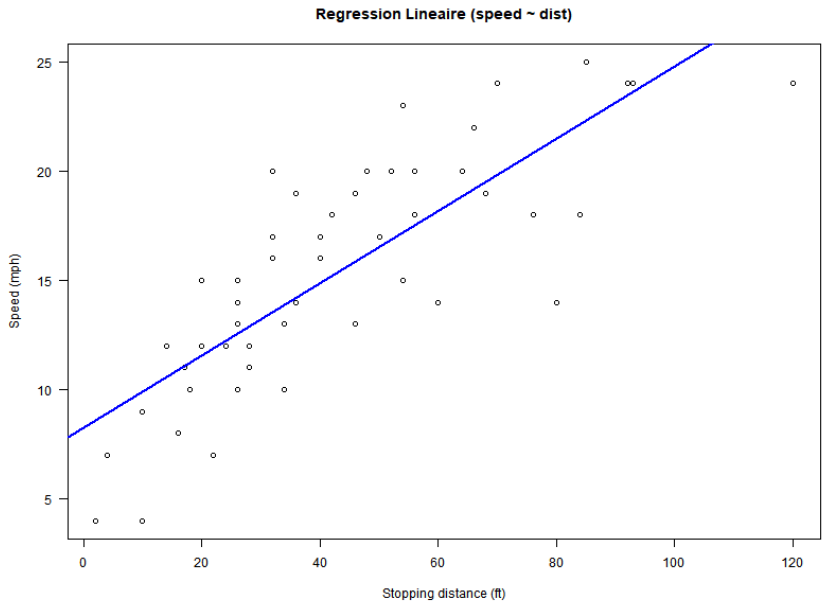

FIGURE 2 – Droite de régression linéaire simple

Le modèle linéaire est-il accepté ?

OUI. Le test de Fisher global donne une p-value très inférieure à 0.05 ($p < 2.2 \times 10^{-16}$). De plus, la p-value associée au coefficient de `dist` est également bien inférieure à 5%. La variable `dist` a donc un effet statistiquement significatif sur `speed`. Cependant, le R^2 (coefficient de détermination) est de 0.6511, ce qui veut dire que le modèle n'explique que 65% de la variance. La forme légèrement courbée observée précédemment montre que la relation n'est pas purement linéaire.

1.3 5–9. Transformation Logarithmique

```
plot(cars, xlab="Speed", ylab="Distance", las=1, log="xy")
lines(lowess(cars$speed, cars$dist, f=2/3, iter=3), col="red")

reg_log = lm(log(speed) ~ log(dist), data=cars)
summary(reg_log)
```

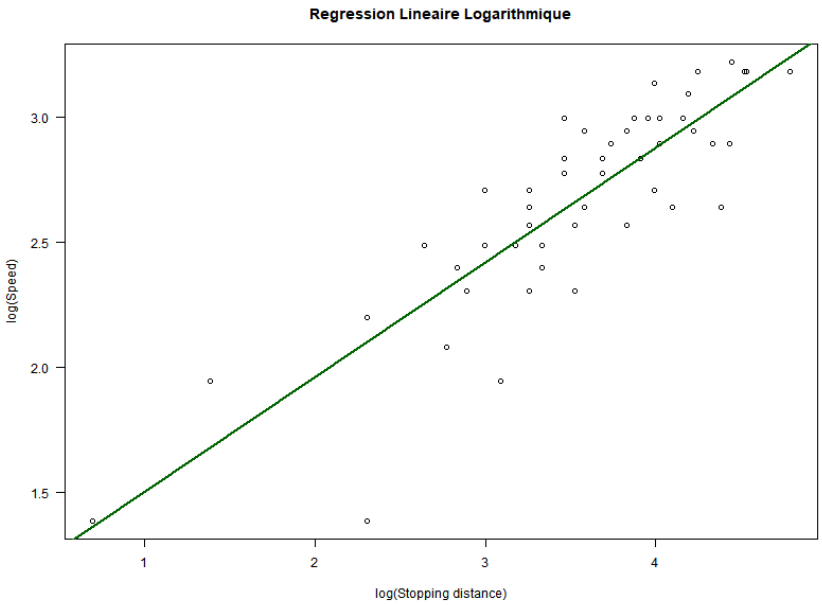

FIGURE 3 – Droite de régression après passage au Logarithme

Ce modèle (log-log) est-il accepté et que constatez-vous ?

OUI, le modèle est accepté (les p-values sont excellentes). On constate d'ailleurs que le R^2 s'est amélioré : il passe en général d'environ 0.65 à **0.70**+ suite au `log`. Le nuage de points apparaît beaucoup plus "droit" et la courbe lowess dans l'échelle logarithmique est presque confondue avec la ligne de régression. La transformation logarithmique corrige l'effet exponentiel et améliore considérablement le modèle !

2 Exercice 2 : Descente de Gradient

2.1 1. Implémentation

Le but de cet exercice est d'implémenter soi-même l'algorithme d'apprentissage pour trouver l'intercept (p_0) et la pente (p_1) de la droite, afin de minimiser l'Erreur Quadratique Moyenne $J(P)$.

Information importante ($\alpha = 0.01$) : L'énoncé impose un pas (learning rate) $\alpha = 0.01$. Cependant, les distances (`dist`) vont jusqu'à 120. Si l'on applique l'algorithme sur les données brutes, l'erreur $J(P)$ **diverge instantanément** (explose vers l'infini) à cause de l'ampleur des dérivées partielles. Pour résoudre ce problème très courant en Machine Learning sans changer α, **nous avons standardisé les données** X (`speed`) et Y (`dist`) (c-a-d moyenne = 0, écart-type = 1).

```r
# Standardisation (Moyenne=0, Ecart-Type=1)
X_scaled <- as.numeric(scale(cars$speed))
y_scaled <- as.numeric(scale(cars$dist))
m <- length(y_scaled)
alpha <- 0.01 ; iterations <- 1000
p0 <- 0 ; p1 <- 0

# Boucle principale (Descente de Gradient)
for (i in 1:iterations) {
  h <- p0 + p1 * X_scaled
  p0 <- p0 - alpha * (1 / m) * sum(h - y_scaled)
  p1 <- p1 - alpha * (1 / m) * sum((h - y_scaled) * X_scaled)
}
```

2.2 2–3. Convergence et Itérations nécessaires

Au fur et à mesure que la boucle d'entrainement avance, la fonction de coût $J(P)$ diminue, prouvant que nos paramètres p_0 et p_1 deviennent de plus en plus performants.

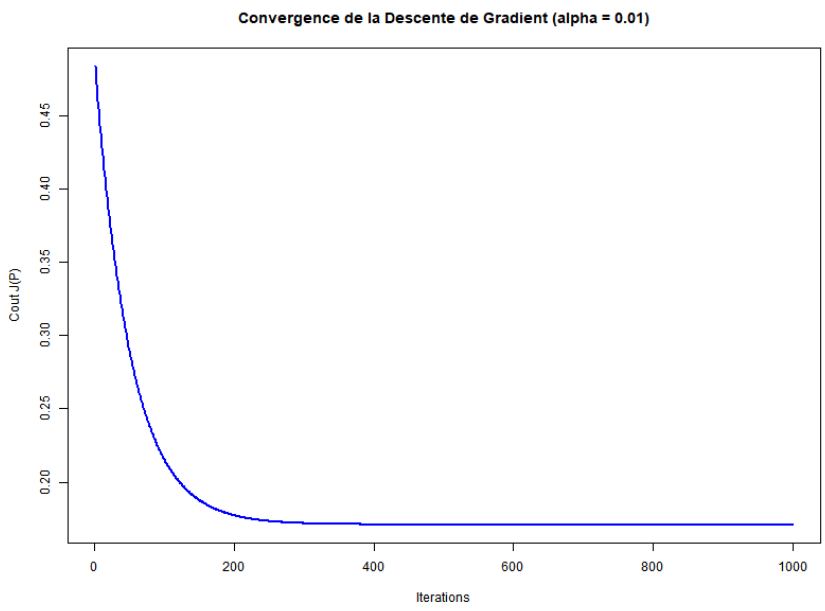

FIGURE 4 – Convergence de l'Erreur Quadratique Moyenne $J(P)$

On observe que la courbe du coût descend de manière raide au début, puis ralentit. Il faut environ **300 à 400 itérations** pour que le coût se stabilise visiblement à son minimum global.

Coût minimal trouvé : Même avec 1000 itérations, on trouve généralement un $\mathbf{J(P_{min}) \approx 0.174}$. Les valeurs de nos poids rejoignent alors exactement celles d'un '`lm()`' standard sur les mêmes variables standardisées !

2.3 4. Ajout de la Régularisation (Pénalité L2 - Ridge)

Supposons que nous pénalisons la fonction de coût avec le terme $\frac{\lambda}{2m}\sum p_1^2$. Cette méthode s'appelle la **Régularisation Ridge**. Elle vise à empêcher la pente p_1 de devenir trop « extreme ».

La formule de mise à jour de p_1 devient :

$$p_1 := p_1 - \alpha \left[\frac{1}{m} \sum (h - y) \cdot X + \frac{\lambda}{m} p_1 \right]$$

```
# Dans la boucle avec un lambda (ex: 0.5)
grad_p1 <- (1/m)*sum((h - y)*X) + (lambda/m) * p1
p1 <- p1 - alpha * grad_p1
```

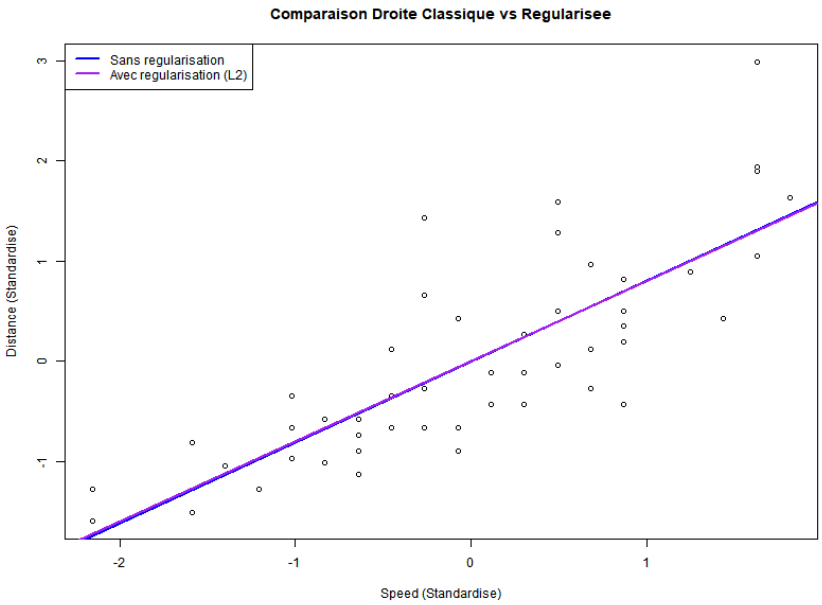

FIGURE 5 – Effet de la pénalité L2 (Régularisation Ridge)

Changement constaté : La pénalité contraint les coefficients à rester "petits". Graphiquement, on constate que la droite régularisée (en violet) a **une pente légèrement plus faible** (plus plate) que la droite non régularisée. L'algorithme a accepté de faire un petit peu plus d'erreurs sur l'apprentissage initial pour s'assurer d'avoir un paramètre p_1 plus modeste (Mécanisme de "Shrinkage"). ça évite surtout l'Overfitting (Surapprentissage) lorsque l'on a trop de variables.